Анализ данных с использованием языка Python

Пояснения к коду. Семинар 11

Проверка нормальности распределения

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-11-normality.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 8–10, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/statistics.md`, раздел «Нормальность».

1 Зачем нормальность и что это такое

Ячейка 1. Подключение, водители и их возраст машины

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  import pandas as pd
4  import seaborn as sns
5  from scipy import stats
6  from statsmodels.stats.diagnostic import lilliefors
7
8  sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
9  dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
   ↪ parse_dates=['datetime'])  # ДТП Брянской области
10 people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # ↩
   ↪ участники ДТП
11 people['experience_clean'] = people['experience'].replace([95, 96, 99], ↩
   ↪ np.nan)  # коды стажа пропусками
12 drivers = people[people['role'] == 'Водитель'].merge(dtp, ↩
   ↪ left_on='dtp_id', right_index=True)  # водители с их ДТП
13 drivers['vehicle_age'] = drivers['datetime'].dt.year - ↩
   ↪ drivers['vehicle_year']  # возраст машины в годах
14 drivers['vehicle_age'] = ↩
   ↪ drivers['vehicle_age'].where(drivers['vehicle_age'] <= 100)  # ↩
   ↪ невозможный возраст в пропуск
15 drivers[['experience_clean', 'vehicle_age']].describe().round(2)
```

– `from scipy import stats` — из библиотеки scipy подключается модуль `stats` со статистическими функциями: нормальное распределение, тесты, Q–Q график. Дальше они вызываются как `stats.shapiro`, `stats.norm.pdf`.

– `from statsmodels.stats.diagnostic import lilliefors` — из библиотеки statsmodels подключается одна функция, тест Лиллиефорса. После такой записи она вызывается просто `lilliefors(...)`, без имени библиотеки.

– Строки 9–13 — таблицы, коды стажа пропуском, водители со своими ДТП и возраст машины, как на семинарах 9 и 10.

– `.where(drivers['vehicle_age'] <= 100)` — там, где возраст не больше 100 лет, значение остаётся, остальное становится пропуском: невозможный возраст. Среди водителей таких нет, но правило страхует от ошибок ввода.

– `describe()` двух столбцов — сводка стажа и возраста машины.

Ячейка 2. Гистограмма стажа и нормальная кривая

```
experience = drivers['experience_clean'].dropna()  # стаж без пропусков
mean, std = experience.mean(), experience.std()  # параметры нормальной кривой
print(round(mean, 2), round(std, 2))
grid = np.linspace(experience.min(), experience.max(), 200)  # точки для кривой
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(x=experience, stat='density', discrete=True, ax=ax)  # гистограмма плотности
ax.plot(grid, stats.norm.pdf(grid, loc=mean, scale=std), color='red')  # кривая нормального распределения
ax.set_title('Стаж водителей и нормальное распределение')
ax.set_xlabel('Стаж, лет')
ax.set_ylabel('Плотность');
```

– `dropna()` — стаж без пропусков. Функции scipy пропуски не пропускают, поэтому чистый столбец берётся сразу.

– Строка 2 — среднее и стандартное отклонение в две переменные одной строкой: они задают колокол.

– `np.linspace(experience.min(), experience.max(), 200)` — 200 равноотстоящих точек от наименьшего стажа до наибольшего; в них считается высота кривой.

– `stat='density'` — столбцы в плотностях: площадь всех столбцов равна единице, как площадь под кривой, и высоты можно сравнивать. `discrete=True` — столбец на каждый целый год.

– `stats.norm.pdf(grid, loc=mean, scale=std)` — высота нормальной кривой в каждой точке сетки. `loc` — среднее, где вершина колокола; `scale` — стандартное отклонение, его ширина.

– `ax.plot(grid, ..., color='red')` — линия по точкам: по горизонтали сетка, по вертикали высоты кривой; красным цветом.

Ячейка 3. Правило 68–95

```
distance = (experience - mean).abs()  # удаление от среднего
print(round((distance <= std).mean(), 3), round((distance <= 2 * std).mean(), 3))  # доли в пределах отклонений
```

– `(experience - mean).abs()` — насколько каждое значение отстоит от среднего; `abs()` — модуль, расстояние без знака.

– `(distance <= std).mean()` — доля значений в пределах одного стандартного отклонения: среднее маски из `True` и `False` — это доля `True`. Так же для двух отклонений, `2 * std`.

Ячейка 4. Асимметрия и эксцесс

```
print(round(experience.skew(), 2), round(experience.kurt(), 2))  # ↩
↪ асимметрия и эксцесс
```

– `skew()` — асимметрия, как на семинаре 10; `kurt()` — эксцесс: около нуля у нормального распределения, больше нуля — хвосты тяжелее нормальных, меньше — легче.

2 Глазами: Q–Q график

Ячейка 5. Q–Q график стажа

```
fig, ax = plt.subplots(figsize=(5, 5))
stats.probplot(experience, plot=ax)  # Q-Q график стажа
ax.set_title('Q-Q график: стаж водителей')
ax.set_xlabel('Квантили нормального распределения')
ax.set_ylabel('Стаж, лет');
```

– `stats.probplot(experience, plot=ax)` — Q–Q график: значения выстраиваются по возрастанию и каждое ставится против места, где оно стояло бы у нормального распределения. `plot=ax` — в какой рамке рисовать. Синие точки — значения, красная прямая — нормальное распределение, подобранное по данным.

– Строки 3–5 — заголовок и подписи осей: scipy ставит их по-английски, здесь они заменены русскими.

Ячейка 6. Q–Q графики трёх столбцов

```
age = drivers['vehicle_age'].dropna()  # возраст машины без пропусков
hour = dtp['datetime'].dt.hour  # час каждого ДТП
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
stats.probplot(experience, plot=axes[0])  # Q-Q график стажа
stats.probplot(age, plot=axes[1])  # Q-Q график возраста машины
stats.probplot(hour, plot=axes[2])  # Q-Q график часа
axes[0].set_title('Стаж водителя')
axes[1].set_title('Возраст машины')
axes[2].set_title('Час ДТП')
for ax in axes:
    ax.set_xlabel('Квантили нормального распределения')
    ax.set_ylabel('Значения')
fig.tight_layout()
```

– `hour = dtp['datetime'].dt.hour` — час из таблицы ДТП, а не водителей: одно ДТП — одно значение; в таблице водителей ДТП с двумя машинами попало бы в счёт дважды.

– Строки 3–6 — три рамки в ряд и Q–Q график в каждой.

– `for ax in axes:` — цикл по трём рамкам: две строки под ним с отступом выполняются для каждой рамки по очереди, и подписи осей ставятся всем трём. Двоеточие в конце строки и отступ обязательны: так Python понимает, какие строки повторять; в курсе отступ — четыре пробела.

Ячейка 7. Ящики с усами трёх столбцов

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3))
sns.boxplot(x=experience, ax=axes[0])  # ящик стажа
sns.boxplot(x=age, ax=axes[1])  # ящик возраста машины
sns.boxplot(x=hour, ax=axes[2])  # ящик часа
axes[0].set_xlabel('Стаж, лет')
axes[1].set_xlabel('Возраст машины, лет')
axes[2].set_xlabel('Час ДТП')
fig.tight_layout()
```

– Три горизонтальных ящика с усами в ряд, как на семинарах 8 и 10.

3 Три теста

Ячейка 8. Тест Шапиро–Уилка на выборке 500

```
sample = experience.sample(500, random_state=1)  # случайные 500 водителей
result = stats.shapiro(sample)  # тест Шапиро-Уилка
result
```

– `experience.sample(500, random_state=1)` — 500 случайных значений без повторов. `random_state=1` — зерно случайности: при одном зерне у всех получаются одни и те же 500 водителей.

– `stats.shapiro(sample)` — тест Шапиро–Уилка. Результат — объект `ShapiroResult` с двумя полями: `statistic`, это W, и `pvalue`.

Ячейка 9. Поля результата и решение словами

```
print(round(result.statistic, 3), result.pvalue)
'отвергаем' if result.pvalue < 0.05 else 'не отвергаем'  # решение при ↩
↪ уровне 0,05
```

– `result.statistic`, `result.pvalue` — поля результата достают через точку.

– `'отвергаем' if условие else 'не отвергаем'` — условное выражение: если условие верно, результат — первое слово, иначе второе. Условие здесь — p меньше уровня значимости 0.05.

Ячейка 10. Тест по всем 14 тысячам значений: ловушка без ошибки

```
stats.shapiro(experience)  # тест по всем значениям
```

– Тест по всем 14 028 значениям. scipy выполняет его, но печатает предупреждение `UserWarning`: при n больше 5 000 p может быть неточным. Ошибки нет, и предупреждение легко пропустить.

Ячейка 11. Тест Д'Агостино

```
stats.normaltest(experience)  # тест Д'Агостино
```

– `stats.normaltest(...)` — тест Д'Агостино: соединяет в одну статистику, насколько асимметрия и эксцесс отличаются от нулей нормального распределения. Рассчитан на большие выборки, поэтому по всем значениям. Результат — объект `NormaltestResult` с полями `statistic` и `pvalue`.

Ячейка 12. Тест Лиллиефорса

```
lilliefors(experience)  # статистика D и p
```

– `lilliefors(...)` — тест Лиллиефорса из statsmodels: сравнивает данные с нормальным распределением, у которого среднее и отклонение взяты из самих данных. Результат — не объект с полями, а два числа в скобках: статистика D и p. Разложить их в две переменные: `d, p = lilliefors(experience)`.

Ячейка 13. Тест Колмогорова–Смирнова без параметров: ловушка без ошибки

```
stats.kstest(experience, 'norm')  # сравнение со стандартным нормальным
```

– `stats.kstest(experience, 'norm')` — сравнение с нормальным распределением со средним 0 и отклонением 1. Стаж в годах с ним не совпадёт никогда, и p получается ноль. Для проверки нормальности — `lilliefors`.

Ячейка 14. Тест и Q–Q график по столбцу с пропусками: ловушка без ошибки

```
gaps = drivers['experience_clean']  # стаж с пропусками
print(stats.shapiro(gaps.sample(500, random_state=1)))  # тест по выборке ↩
↪ с пропусками
fig, ax = plt.subplots(figsize=(5, 5))
stats.probplot(gaps, plot=ax)  # Q-Q график с пропусками
ax.set_title('Q-Q график по столбцу с пропусками');
```

– `gaps` — столбец стажа как есть, с пропусками.

– Строка 2 — тест по выборке 500 из столбца с пропусками: в выборку попали пропуски, и тест вернул `nan` вместо чисел.

– Строка 4 — Q–Q график по столбцу с пропусками: прямая не нарисована, точки обрываются. Перед любой функцией scipy — `dropna()`.

Ячейка 15. Выборка 500 из маленькой группы: намеренная ошибка

```
snow = drivers.loc[drivers['weather'] == 'Снегопад', 'experience_clean'].dropna()  # стаж в снегопад
stats.shapiro(snow.sample(500, random_state=1))  # тест на выборке 500
```

– Строка 1 — стаж водителей в ДТП во время снегопада, без пропусков: `loc` с маской и именем столбца, как на семинаре 10.

– `snow.sample(500, ...)` — в группе меньше 500 значений, и выборку без повторов такого размера не взять: `ValueError: Cannot take a larger sample than population when 'replace=False'`. Ошибка намеренная.

Ячейка 16. Тест по всей группе

```
print(len(snow))
stats.shapiro(snow)  # тест по всей группе
```

– `len(snow)` — 392 значения: не больше 5 000, поэтому по правилу курса тест идёт по всем, без выборки.

Ячейка 17. Тест на 30 водителях

```
stats.shapiro(experience.sample(30, random_state=1))  # тест на 30 водителях
```

– Выборка из 30 значений с тем же зерном и тест по ней: p 0.167, нормальность не отвергнута. На малой выборке тест отклонения не видит.

4 Центральная предельная теорема

Ячейка 18. Тысяча средних по 100 водителей

```
rng = np.random.default_rng(1)  # генератор с зерном 1
means = rng.choice(experience.to_numpy(), size=(1000, 100)).mean(axis=1)  # 1000 средних по 100
print(round(pd.Series(means).skew(), 2), round(stats.shapiro(means).pvalue, 3))  # асимметрия и p средних
print(round(means.std(), 3), round(std / np.sqrt(100), 3))  # разброс средних и $\sigma/\sqrt{n}$
```

– `np.random.default_rng(1)` — генератор случайных чисел с зерном 1, как на семинаре 3.

– `experience.to_numpy()` — значения стажа массивом NumPy.

– `rng.choice(..., size=(1000, 100))` — таблица 1 000 строк на 100 столбцов из случайных значений стажа; значения могут повторяться. Каждая строка — одна случайная сотня водителей.

– `.mean(axis=1)` — среднее каждой строки: 1 000 средних. `axis=1` — считать вдоль строки, по столбцам.

– `pd.Series(means).skew()` — у массива NumPy нет метода `skew`, поэтому средние на время оборачиваются в Series.

– Строка 4 — разброс 1 000 средних и расчёт по теореме: стандартное отклонение стажа, делённое на корень из 100, `np.sqrt(100)`.

Ячейка 19. Гистограмма средних и кривая теоремы

```
grid_means = np.linspace(means.min(), means.max(), 200)  # точки для кривой
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(x=means, stat='density', ax=ax)  # гистограмма средних
ax.plot(grid_means, stats.norm.pdf(grid_means, loc=mean, scale=std / ↩
↪ np.sqrt(100)), color='red')  # кривая по ЦПТ
ax.set_title('Средний стаж 1 000 выборок по 100 водителей')
ax.set_xlabel('Средний стаж выборки, лет')
ax.set_ylabel('Плотность');
```

– Устройство как в ячейке 2: гистограмма в плотностях и кривая по 200 точкам. `discrete=True` здесь нет: средние дробные.

– `scale=std / np.sqrt(100)` — ширина кривой по теореме: разброс средних, а не отдельных значений.

5 Преобразования и что дальше

Ячейка 20. Корень из возраста машины и тест

```
root = np.sqrt(age)  # корень из возраста
print(round(age.skew(), 2), round(root.skew(), 2))  # асимметрия до и после
stats.shapiro(root.sample(500, random_state=1))  # тест после корня
```

– `np.sqrt(age)` — квадратный корень из каждого значения; результат остаётся Series, поэтому у него есть `skew()` и `sample`.

– Строка 3 — значений 14 784, больше 5 000, поэтому тест на выборке 500.

Ячейка 21. Три гистограммы: как есть, логарифм, корень

```
logged = np.log1p(age)  # логарифм возраста плюс один
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(x=age, discrete=True, ax=axes[0])  # возраст как есть
sns.histplot(x=logged, bins=20, ax=axes[1])  # после логарифма
sns.histplot(x=root, bins=20, ax=axes[2])  # после корня
axes[0].set_title(f'Как есть: асимметрия {age.skew():.2f}')
axes[1].set_title(f'log1p: асимметрия {logged.skew():.2f}')
axes[2].set_title(f'Корень: асимметрия {root.skew():.2f}')
for ax in axes:
    ax.set_xlabel('Значение')
    ax.set_ylabel('Число водителей')
fig.tight_layout()
```

– `np.log1p(age)` — логарифм от значения плюс один. Обычный логарифм нуля — минус бесконечность, а возраст машины бывает нулевым.

– `bins=20` — после преобразования значения дробные, поэтому не столбец на каждое значение, а 20 корзин.

– `f'Как есть: асимметрия {age.skew():.2f}'` — f-строка: буква `f` перед кавычкой, и вместо фигурных скобок подставляется значение выражения внутри них. `:.2f` — два знака после точки.

– Строки 9–11 — цикл по трём рамкам, как в ячейке 6.

6 Раздел «Нормальность» этапа 3

Ячейка 22. Таблица нормальности трёх столбцов

```
three = {'стаж водителя': experience, 'возраст машины': age, 'час ДТП': hour}  # три столбца
table = pd.DataFrame({'n': [values.count() for values in three.values()],
                      'асимметрия': [round(values.skew(), 2) for values in three.values()],
                      'тест': 'Д'Агостино',
                      'p': [stats.normaltest(values).pvalue for values in three.values()]}, index=list(three))
table['решение'] = np.where(table['p'] < 0.05, 'отвергаем', 'не отвергаем')  # решение при 0,05
small = np.where(table['решение'] == 'отвергаем', 'непараметрический тест',
                 'мало данных: решать по Q-Q графику и асимметрии')  # если значений меньше 30
table['что дальше'] = np.where(table['n'] >= 30, 't-тест', small)  # сначала размер выборки
table
```

– `three = {...}` — словарь «название — столбец»: три Series под русскими именами.

– `[values.count() for values in three.values()]` — список, собранный циклом в одну строку: для каждого Series из словаря `values.count()`, число значений без пропусков. Так же собираются асимметрии и p.

– `'тест'` и одна строка «Д'Агостино» на все строки таблицы: pandas повторяет её в каждой.

– `index=list(three)` — строки таблицы подписаны ключами словаря, названиями столбцов.

– `np.where(маска, 'отвергаем', 'не отвергаем')` — по маске выбирается одно из двух слов, как на семинаре 3.

– Строки 7–9 — правило «что дальше» двумя `np.where`: сначала `small`, слово для малой выборки по решению теста, потом t-тест или это слово по размеру выборки.